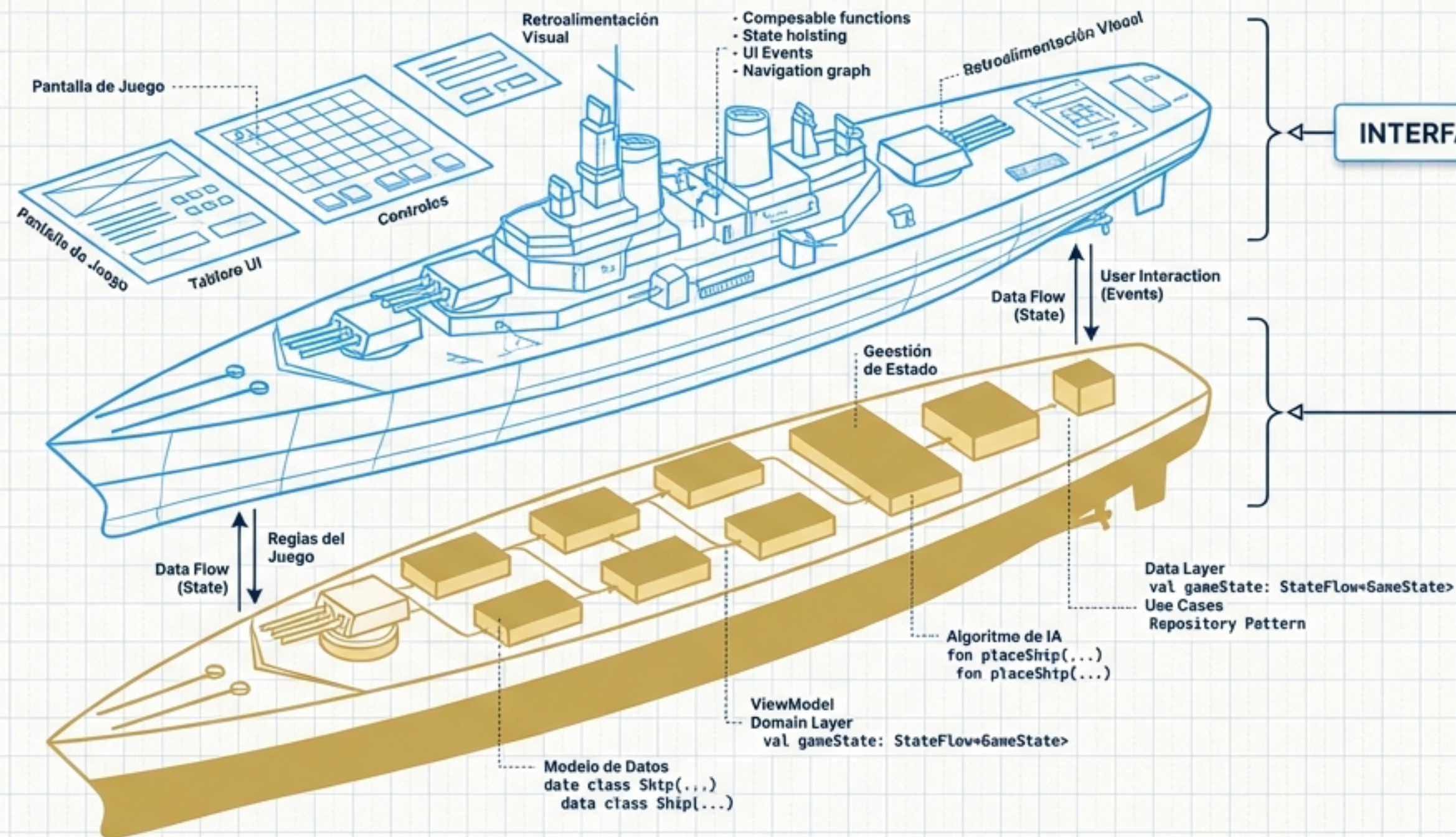


• Hundir la Flota: Anatomía de una App en Android •

- Entendiendo la separación entre la Lógica y la Interfaz usando Kotlin y Jetpack Compose.



Dividiendo el Mundo en Dos Hemisferios

El Cerebro (Lógica de Negocio)

`Juego.kt`

Las matemáticas, las reglas y los datos. Es puro Kotlin. No sabe absolutamente nada sobre pantallas, colores o botones.



El Lienzo (Interfaz de Usuario)

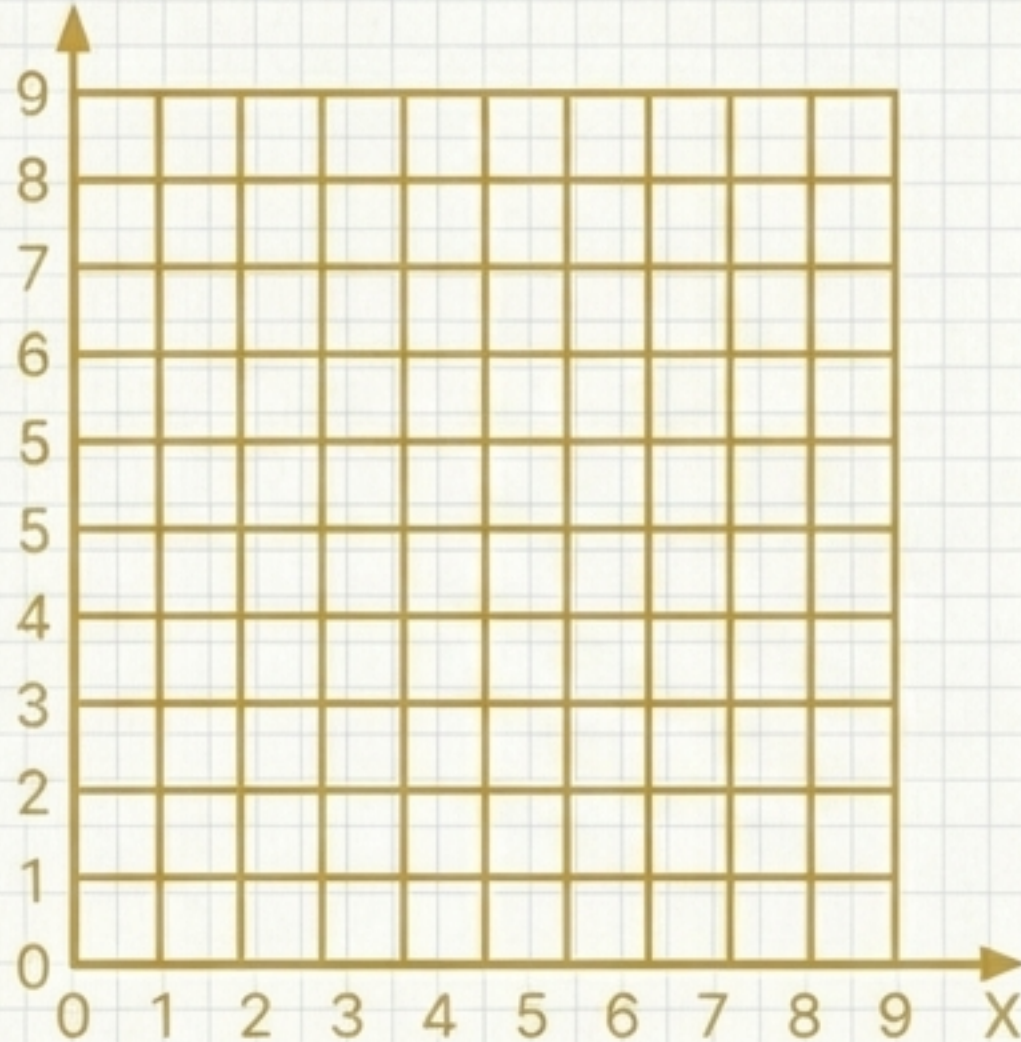
`MainActivity.kt`

Lo que el usuario ve y toca. Construido con Jetpack Compose. Dibuja el estado actual y captura los toques del jugador.



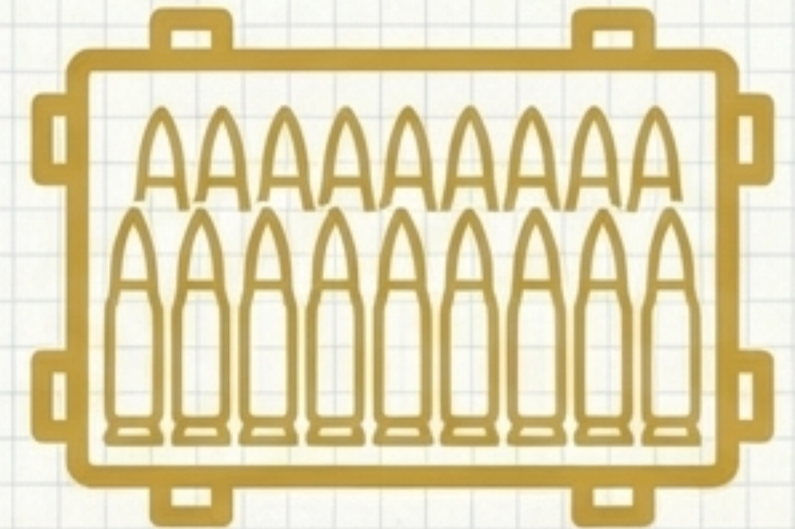
Las Leyes de la Física del Juego

Define la realidad espacial.
Un solo número entero crea automáticamente un tablero de 100 posiciones.



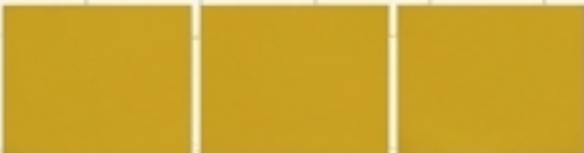
```
val DIMENSION = 10  
val MUNICION_MAXIMA = 50
```

Define la restricción del jugador (se requieren mínimo 17 aciertos para hundir la flota completa).



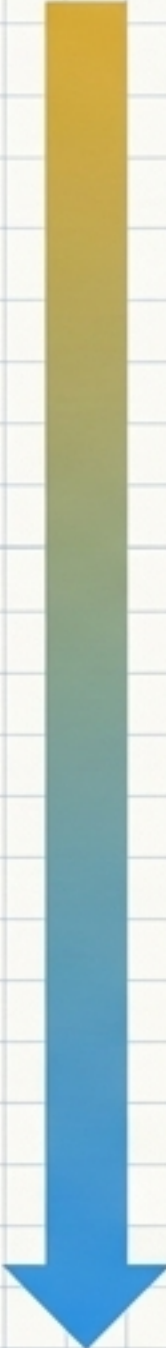
Define la restricción del jugador (se requieren mínimo 17 aciertos para hundir la flota completa).

El Manifiesto de la Flota

Clase de Barco	Longitud (val longitud: Int)	Representación en Cuadrículas
PORTAAVIONES JetBrains Mono	5 JetBrains Mono	
ACORAZADO JetBrains Mono	4 JetBrains Mono	
CRUCERO JetBrains Mono	3 JetBrains Mono	
SUBMARINO JetBrains Mono	3 JetBrains Mono	
DESTRUCTOR JetBrains Mono	2 JetBrains Mono	

El enum `class TipoBarco` asegura que la aplicación nunca pueda crear un barco de un tamaño inválido por accidente.

[El Lenguaje Visual del Tablero]

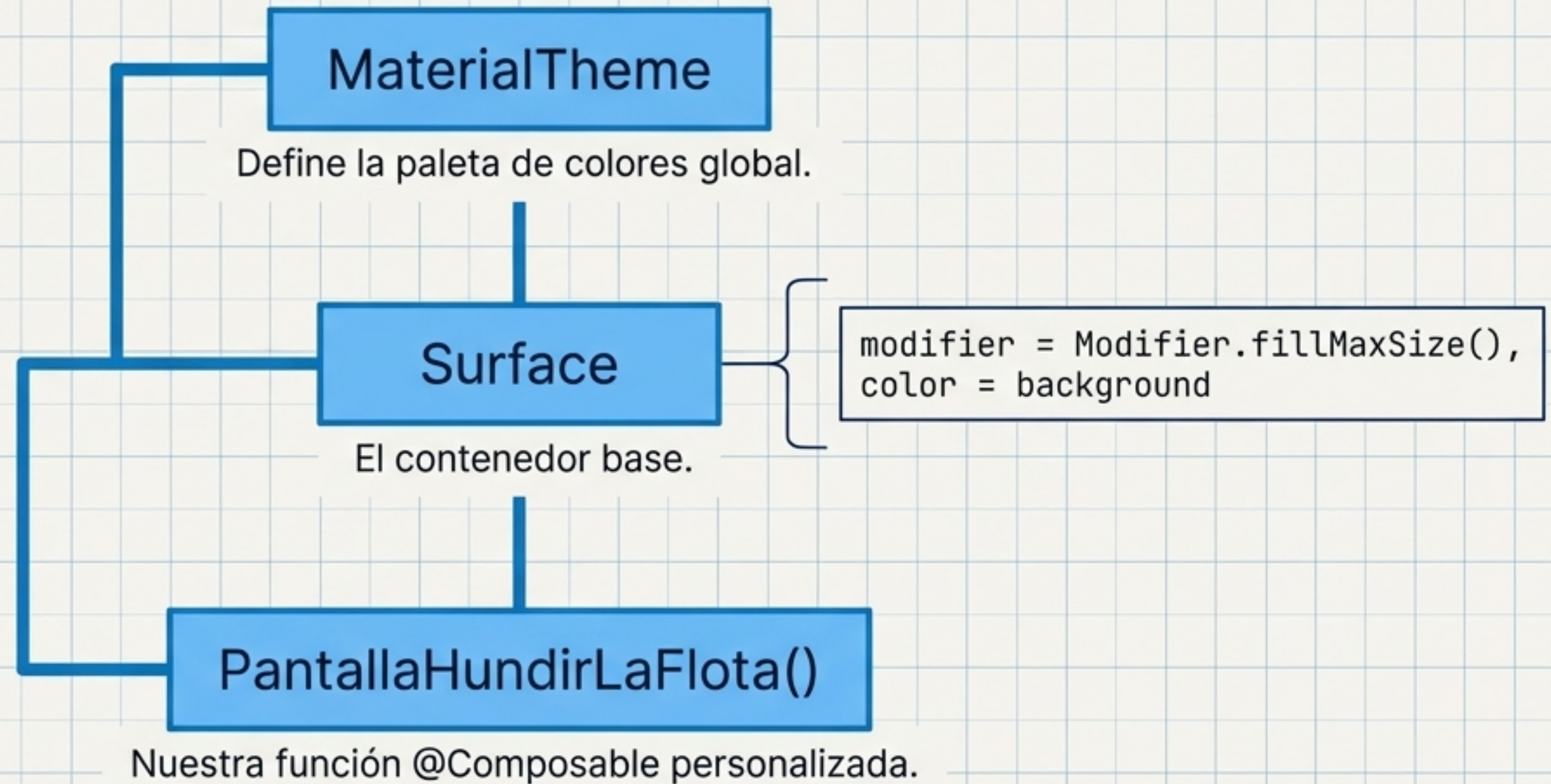


Estado Interno (enum)	Símbolo en Pantalla	Significado Táctico
AGUA		Océano inexplorado.
BARCO		Posición secreta de la flota (Oculto al jugador durante la partida).
TOCADO		Impacto confirmado. Munición reducida.
FALLO		Disparo desperdiciado en el agua.

MainActivity: El Portal al Sistema Operativo



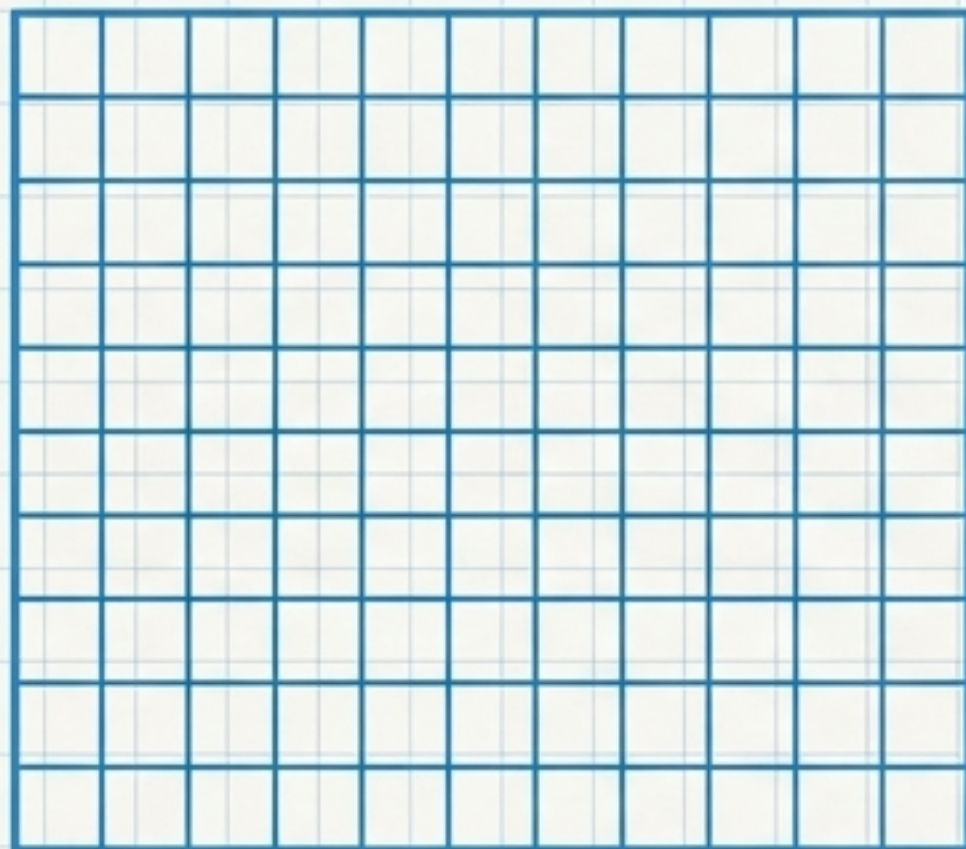
El Árbol de Jerarquía Visual



Las interfaces modernas no se dibujan; se ensamblan anidando bloques de construcción independientes.

El Problema de la Amnesia en la Interfaz

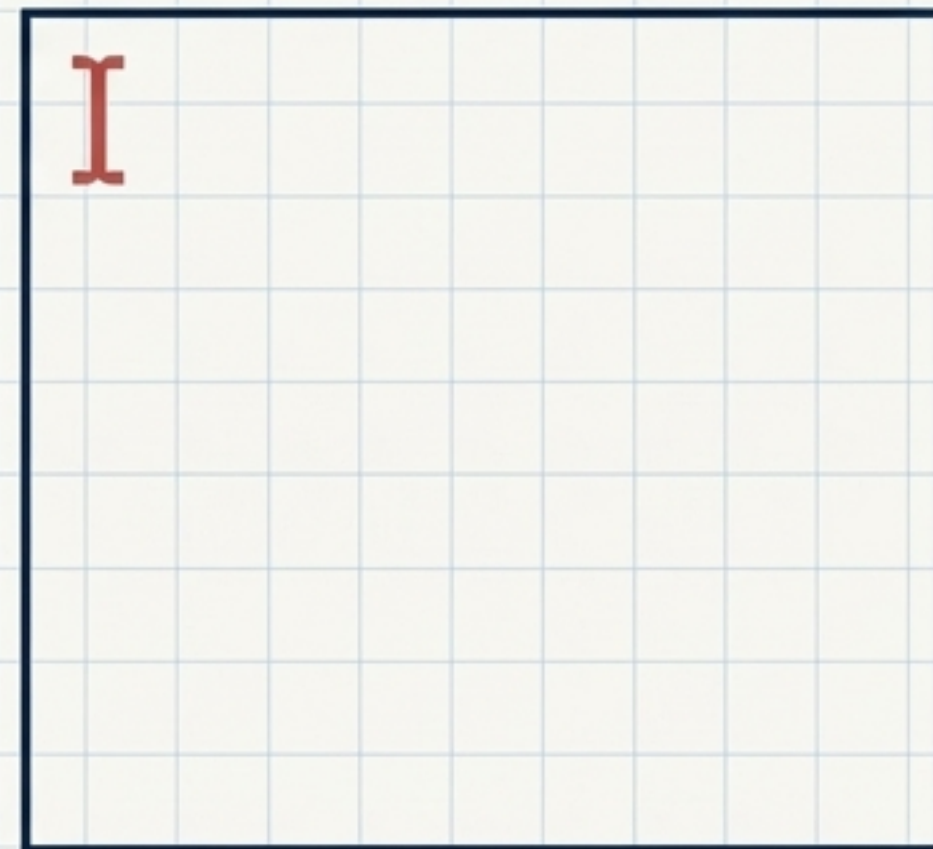
1. Pantalla dibujada



2. El usuario toca una casilla



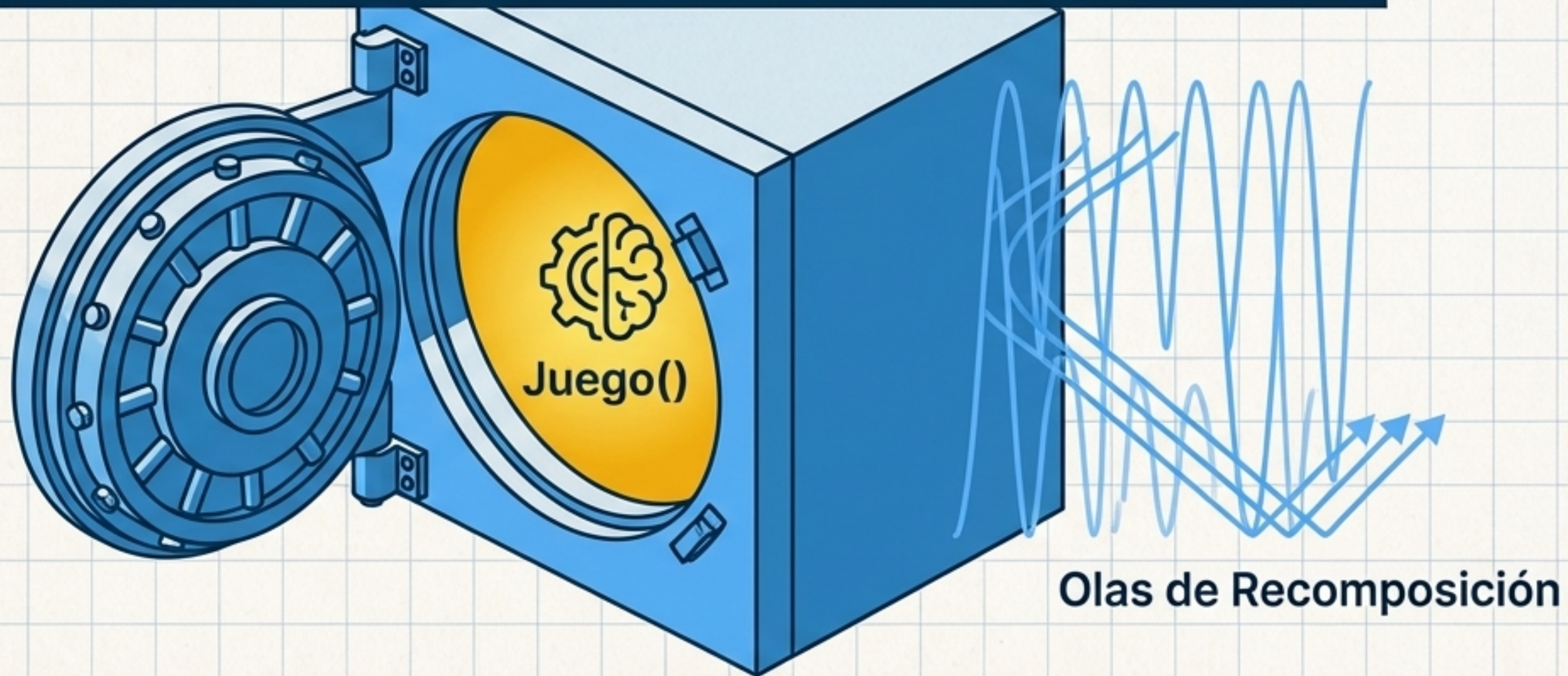
3. La pantalla se reinicia



Las funciones `@Composable` se redibujan constantemente. Si declaramos nuestro `Juego()` directamente aquí, cada toque reiniciaría la partida desde cero. La interfaz no tiene memoria propia.

Salvaguardando el Cerebro: remember

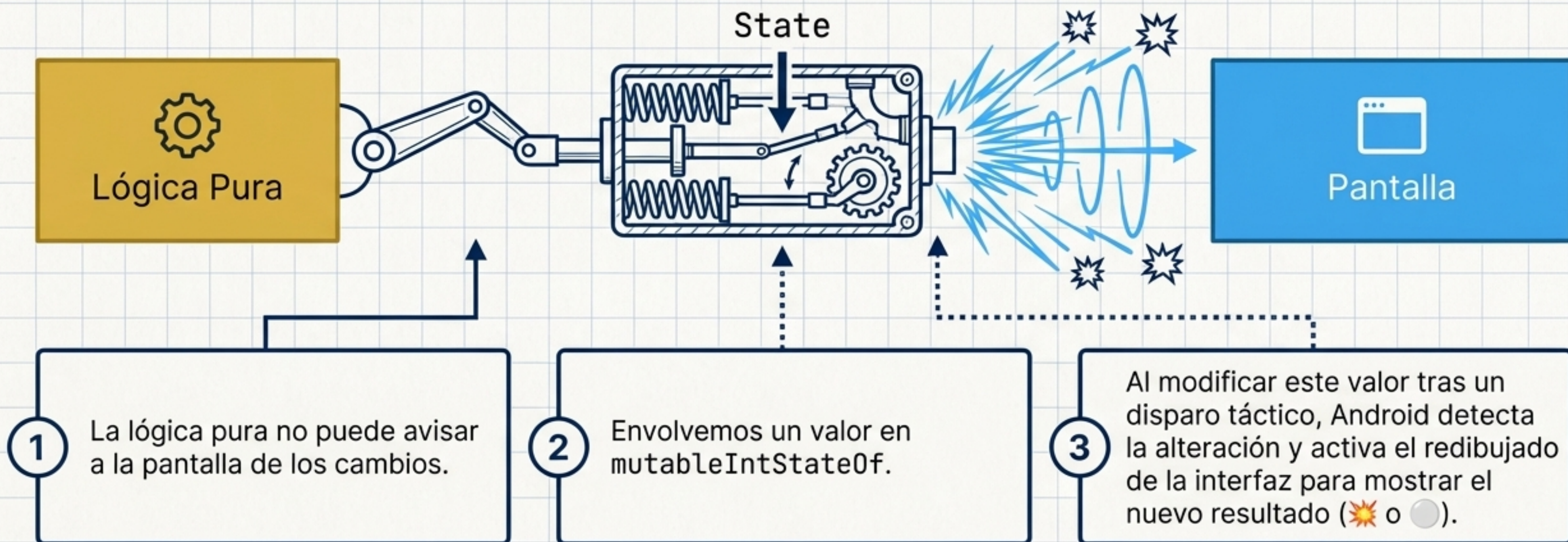
```
val motorJuego = remember { Juego() }
```



remember le dice a Android: "Guarda esta instancia de la lógica de negocio en una caja fuerte. Cuando la pantalla se redibuje, no crees un juego nuevo, saca el que guardamos aquí."

El Disparador de Cambios: El Estado

```
var refrescar by remember { mutableIntStateOf(0) }
```



El Bucle Reactivo en Acción

